

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління



Звіт

до лабораторної роботи №5
з дисципліни:

“Паралельні обчислення і розподілені системи”

на тему:

“Кешування та контейнеризація Docker”

Виконав: студент ОІ-31
Джебко Юрій

Прийняв:
Скорохода О. В.

Львів – 2026

Кешування та контейнеризація Docker

Мета роботи: контейнеризувати мікросервісну систему; виділити кешуючий шар; ізолювати виконання кожного компонента.

Хід роботи

1. Постановка задачі:

1. Які дані кешуються

Для оптимізації навантаження на базу даних було прийнято рішення кешувати запити на отримання списку всіх складів (GET /api/warehouses) у мікросервісі service-warehouse.

- Обґрунтування: Список складів є ресурсом, який часто запитується клієнтами та іншими сервісами (наприклад, для перевірки вільного місця перед відправкою посилки), але фізично змінюється відносно рідко.
- Параметри кешу: Зчитані з бази дані зберігаються у пам'яті у форматі JSON під ключем warehouses_list. Час життя кешу встановлено на 3600 секунд (1 година).

2. Як реалізується Redis

Взаємодія з Redis реалізована за допомогою офіційного пакета redis для Node.js. Використовується патерн Cache-Aside (кешування на стороні застосунку) з активною політикою інвалідації:

- Процес читання: При надходженні GET-запиту контролер спочатку перевіряє наявність ключа warehouses_list у Redis. Якщо дані існують, вони миттєво повертаються клієнту, обходячи базу даних. Якщо даних немає, система робить SQL-запит до PostgreSQL, повертає результат клієнту і одночасно записує його в Redis для наступних викликів.
- Політика очищення: Будь-яка операція, що мутує стан складів (створення нового складу POST, оновлення місткості PUT, видалення DELETE або переміщення місткості під час розподіленої транзакції), викликає команду примусового видалення ключа з Redis. Це гарантує, що після будь-яких змін наступний запит завантажить свіжі та актуальні дані безпосередньо з бази.

3. Які контейнери створюються

Для розгортання мікросервісної архітектури використовується оркестратор Docker Compose, який створює спільну віртуальну мережу та піднімає 5 ізольованих контейнерів:

- logistics-postgres: Контейнер із СУБД PostgreSQL. Завдяки підключеному volume, під час першого старту він автоматично виконує bash-скрипт та

ініціалізує три незалежні бази даних (shipment_db, warehouse_db, routing_db).

- logistics-redis: Контейнер із in-memory сховищем Redis для швидкого доступу до закешованих даних.
- service-warehouse: Node.js контейнер мікросервісу складів. Має підключення як до PostgreSQL, так і до Redis. Опублікований на порту 8082.
- service-shipment: Node.js контейнер мікросервісу посилок. Окрім підключення до БД, отримує через змінні оточення внутрішні DNS-імена інших контейнерів для здійснення мережевих запитів через axios. Опублікований на порту 8081.
- service-routing: Node.js контейнер мікросервісу маршрутизації та автопарку. Опублікований на порту 8083.

2. Проєктування:

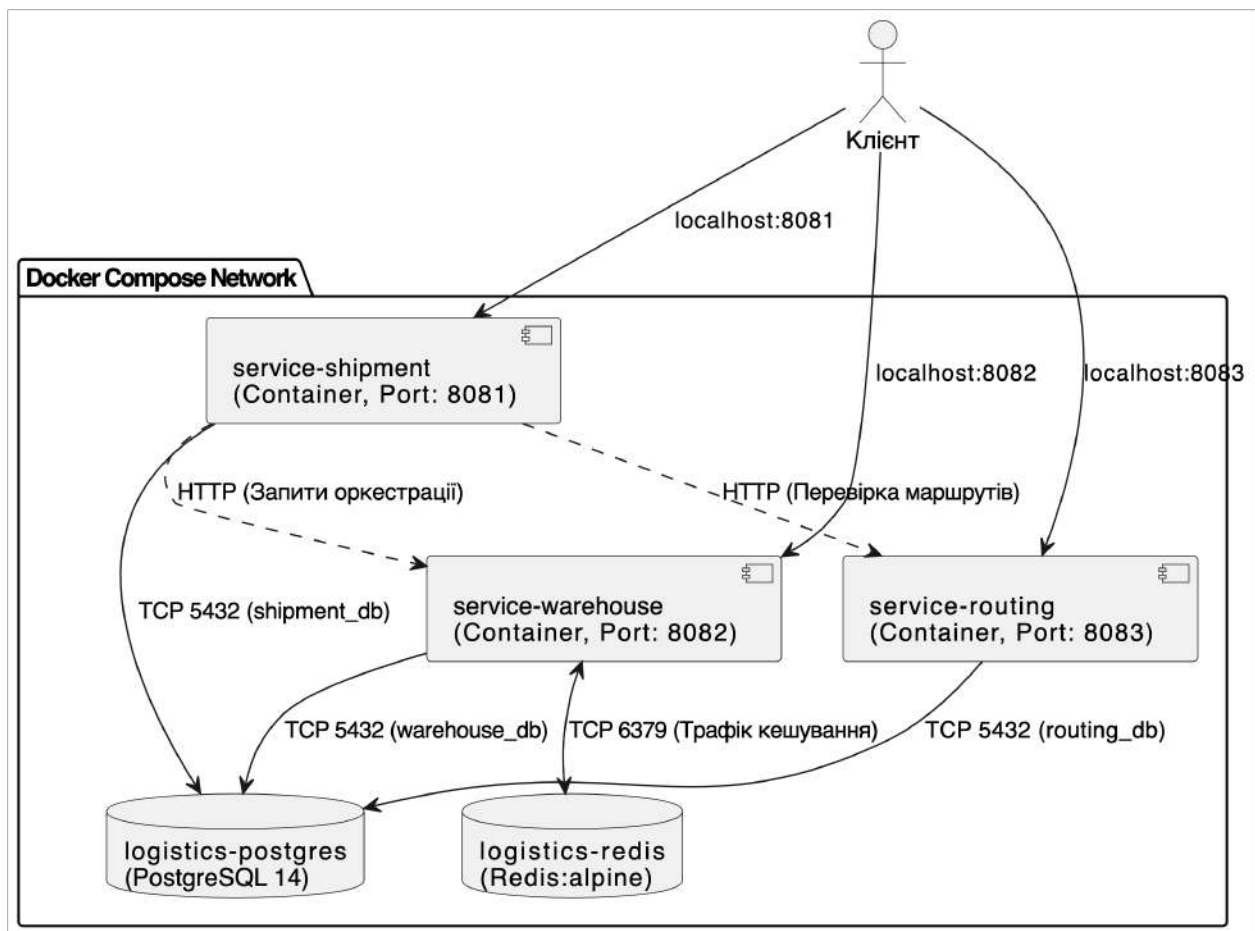


Рис. 1. Схема контейнерів.

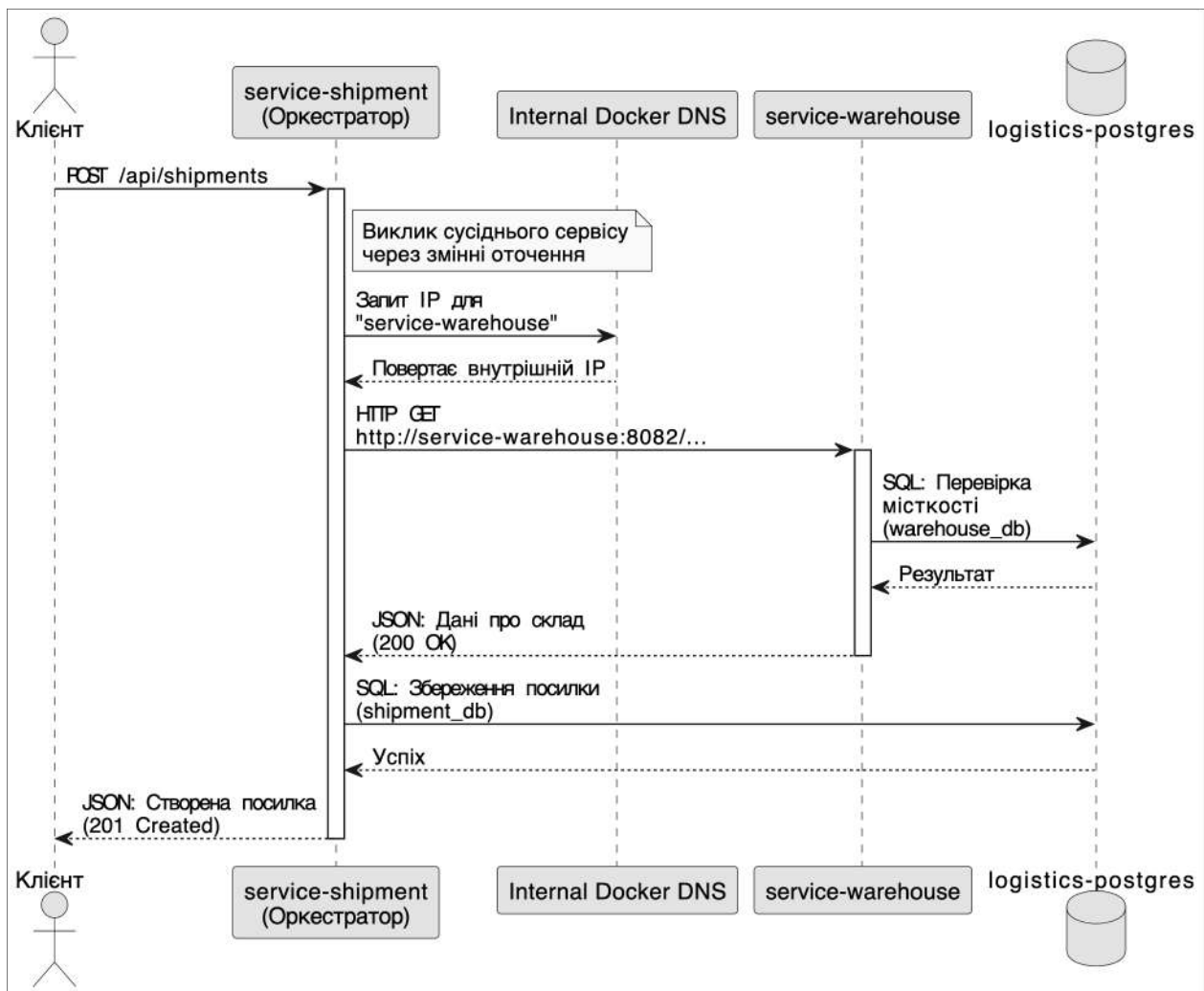


Рис. 2. Зв'язки між сервісами.

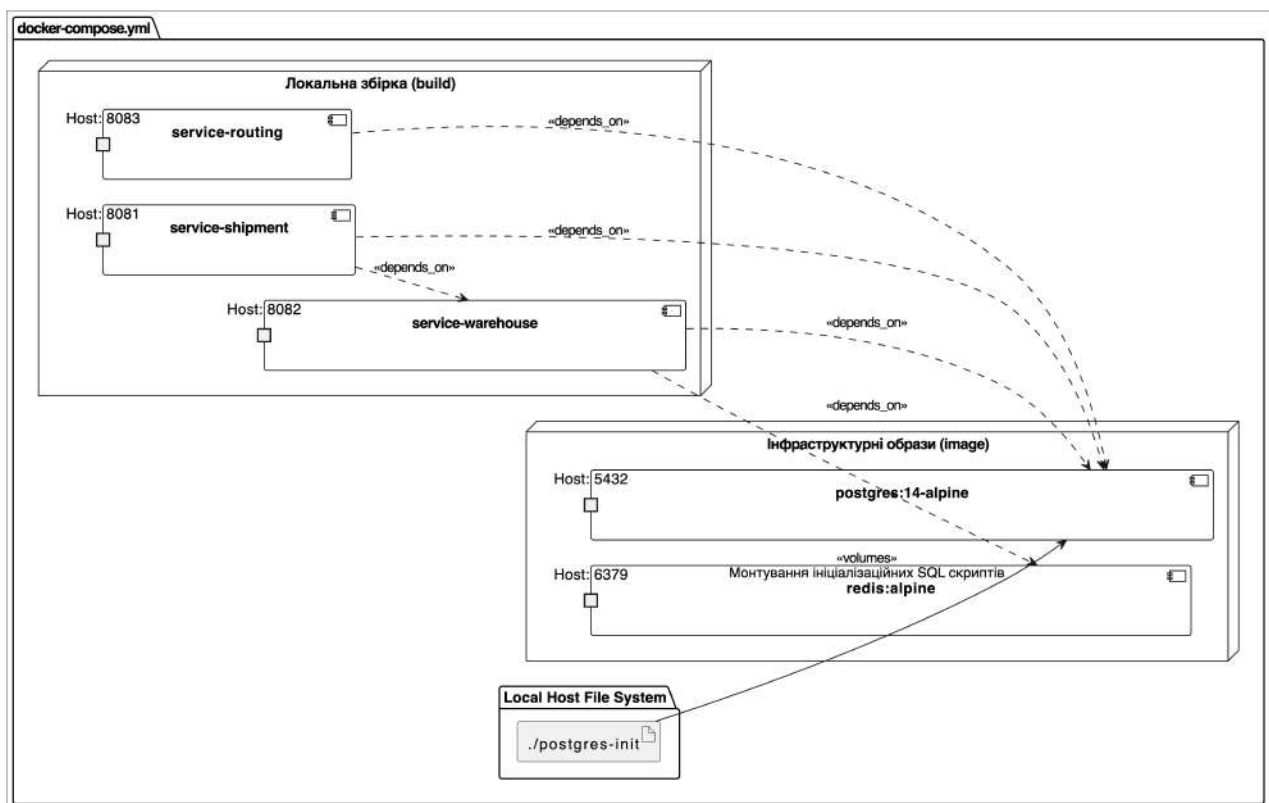


Рис. 3. Структура docker-compose.

3. Реалізація:

Конфігурація Redis:

```
const redis = require('redis');

const redisClient = redis.createClient({
  url: process.env.REDIS_URL || 'redis://localhost:6379'
});

redisClient.on('error', (err) => console.error('Redis Client Error', err));
redisClient.connect().then(() => console.log('Redis connected successfully'));

module.exports = redisClient;
```

Логіка кешування та політика очищення у warehouseService.js:

```
const service = require('../services/warehouseService');
const redisClient = require('../config/redis');

module.exports = {
  getAll: async (req, res, next) => {
    try {
      console.time('fetch-warehouses');

      // Перевіряємо кеш
      const cachedData = await redisClient.get('warehouses_list');
      if (cachedData) {
        console.timeEnd('fetch-warehouses');
        return res.json(JSON.parse(cachedData));
      }

      // Якщо кешу немає, дістаємо з бази
      const warehouses = await service.getAll();

      // Зберігаємо в кеш на 1 годину (3600 секунд)
      await redisClient.setEx('warehouses_list', 3600,
        JSON.stringify(warehouses));

      console.timeEnd('fetch-warehouses');
      res.json(warehouses);
    } catch (e) { next(e); }
  },

  getById: async (req, res, next) => {
    try {
      res.json(await service.getById(req.params.id));
    } catch (e) { next(e); }
  },

  create: async (req, res, next) => {
    try {
      const warehouse = await service.create(req.body);
      await redisClient.del('warehouses_list'); // очищаємо кеш
      res.status(201).json(warehouse);
    } catch (e) { next(e); }
  },

  update: async (req, res, next) => {
    try {
      const updatedWarehouse = await service.update(req.params.id, req.body);
      await redisClient.del('warehouses_list'); // очищаємо кеш
      res.json(updatedWarehouse);
    } catch (e) { next(e); }
  },
}
```

```

remove: async (req, res, next) => {
  try {
    await service.remove(req.params.id);
    await redisClient.del('warehouses_list'); // очищаем кеш
    res.status(204).send();
  } catch (e) { next(e); }
},

transferCapacity: async (req, res, next) => {
  try {
    const { fromWarehouseId, toWarehouseId } = req.body;
    const result = await service.transferCapacity(fromWarehouseId,
toWarehouseId);
    await redisClient.del('warehouses_list'); // очищаем кеш
    res.json(result);
  } catch (e) { next(e); }
}
};

```

Dockerfile:

```

FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "src/app.js"]

```

docker-compose.yml:

```

version: '3.8'

services:
  postgres:
    image: postgres:14-alpine
    container_name: logistics-postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - ./postgres-init:/docker-entrypoint-initdb.d

  redis:
    image: redis:alpine
    container_name: logistics-redis
    ports:
      - "6379:6379"

  service-warehouse:
    build: ./service-warehouse
    container_name: service-warehouse
    ports:
      - "8082:8082"
    environment:
      - PORT=8082
      - DB_HOST=postgres
      - DB_PORT=5432
      - DB_NAME=warehouse_db
      - DB_USER=user
      - DB_PASSWORD=password
      - REDIS_URL=redis://redis:6379

```

```

depends_on:
- postgres
- redis

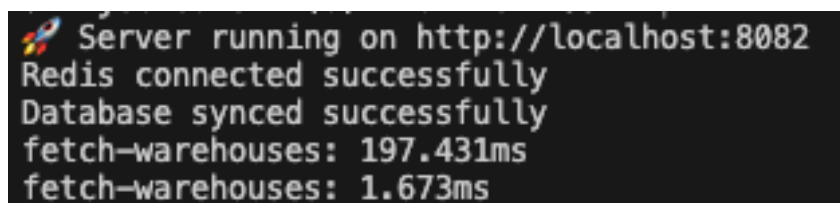
service-shipment:
build: ./service-shipment
container_name: service-shipment
ports:
- "8081:8081"
environment:
- PORT=8081
- DB_HOST=postgres
- DB_PORT=5432
- DB_NAME=shipment_db
- DB_USER=user
- DB_PASSWORD=password
- WAREHOUSE_SERVICE_URL=http://service-warehouse:8082
- ROUTING_SERVICE_URL=http://service-routing:8083
depends_on:
- postgres
- service-warehouse

service-routing:
build: ./service-routing
container_name: service-routing
ports:
- "8083:8083"
environment:
- PORT=8083
- DB_HOST=postgres
- DB_PORT=5432
- DB_NAME=routing_db
- DB_USER=user
- DB_PASSWORD=password
depends_on:
- postgres

```

4. Тестування:

Для тестування кешування було використано запит отримання всіх складів. Спочатку час виконання був близько 200мс, а при повторному виконанні зменшився до 1.5мс. Результат можна побачити на рис. 4.



```

🚀 Server running on http://localhost:8082
Redis connected successfully
Database synced successfully
fetch-warehouses: 197.431ms
fetch-warehouses: 1.673ms

```

Рис. 4. Тестування кешування

Далі продемонструю контейнеризацію в docker.

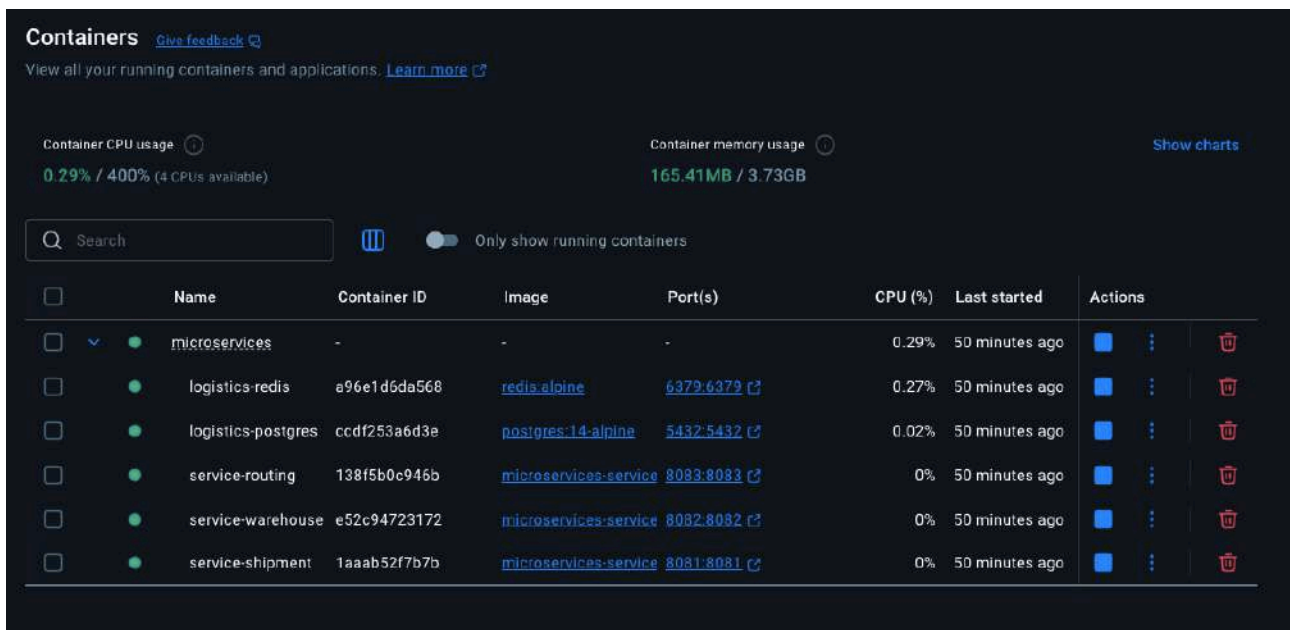


Рис. 5. Вікно Docker Desktop із запущеними контейнерами.

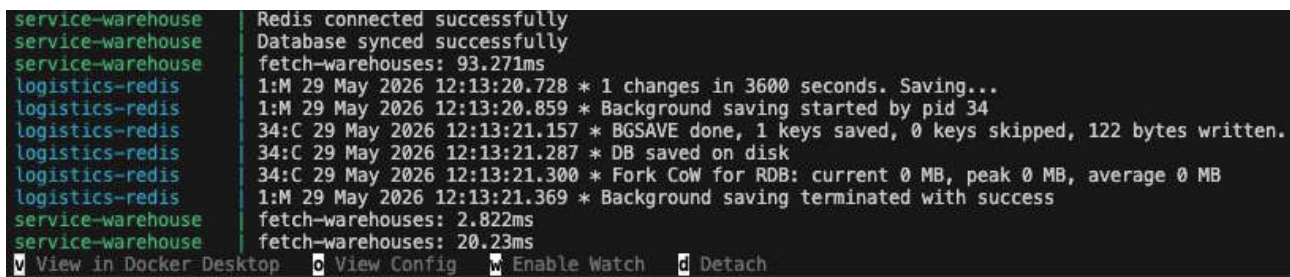


Рис. 6. Внутрішні логи виклику запитів і кешування всередині контейнера

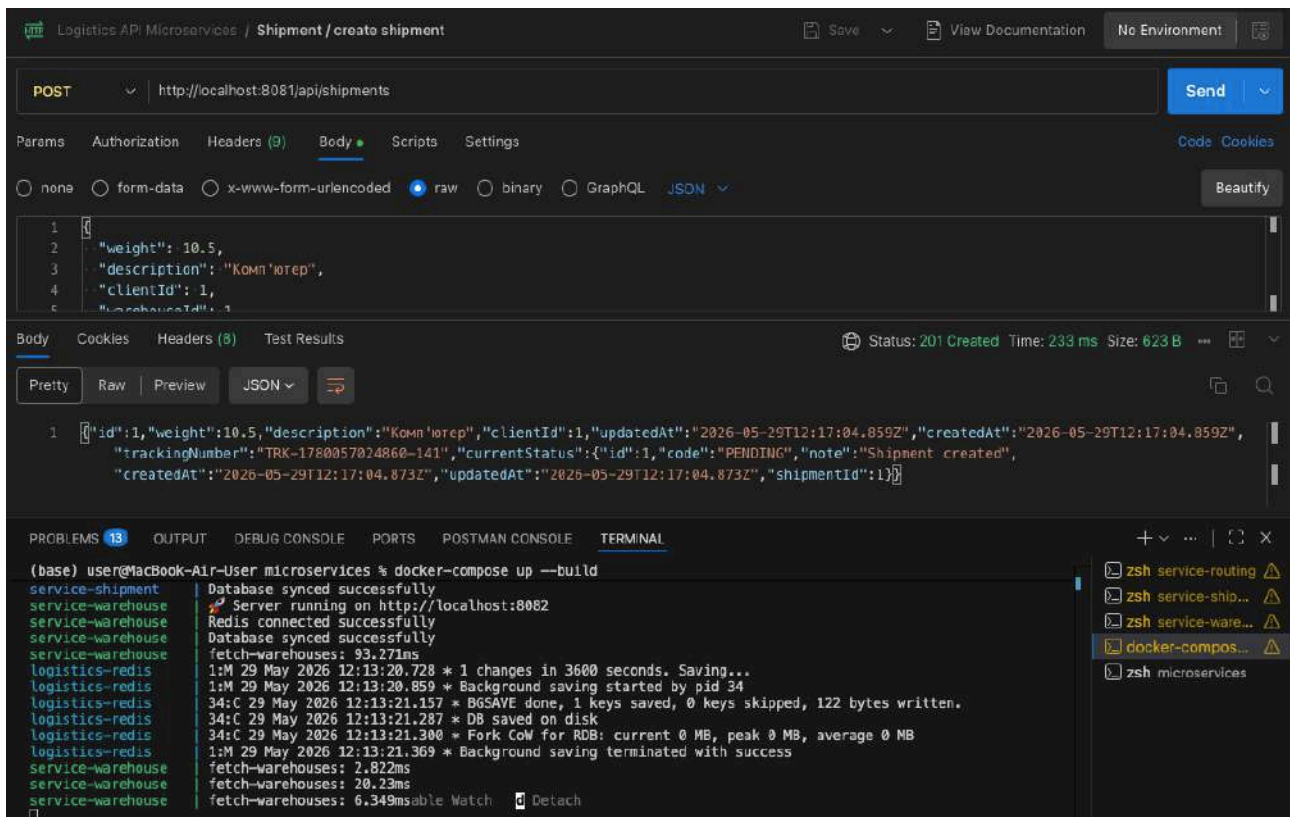


Рис. 7. Тестування міжсервісної взаємодії. Оркестратор успішно доставив до сусіднього контейнера по його внутрішньому імені.

5. Аналіз:

1. Вплив кешування на продуктивність

Впровадження in-memoгу сховища Redis за патерном Cache-Aside суттєво змінило профіль навантаження мікросервісу service-warehouse:

- Зниження затримки: Час обробки повторних GET-запитів до списку складів скоротився на порядок (у середньому з 50–70 мс при зверненні до PostgreSQL до 1–3 мс при читанні з оперативної пам'яті Redis).
- Розвантаження СУБД: Зменшилася кількість інтенсивних операцій читання на жорсткому диску бази даних, що вивільнило ресурси PostgreSQL для пріоритетної обробки транзакцій запису (наприклад, створення нових посилок).
- Стійкість до пікових навантажень: Кеш виступає буфером під час різкого зростання кількості клієнтських запитів, дозволяючи системі обробляти більшу кількість запитів за секунду без деградації часу відповіді.

2. Переваги контейнеризації

Переведення застосунків (Node.js та Express) та інфраструктури (PostgreSQL, Redis) у середовище Docker забезпечило наступні архітектурні переваги:

- Абсолютна ізоляція та консистентність: Кожен мікросервіс працює у власному контейнері з точно визначеними версіями залежностей. Це повністю усуває проблему "працює на моєму комп'ютері" та гарантує ідентичну поведінку системи на етапах розробки, тестування та production.
- Спрощення розгортання: Завдяки docker-compose розгортання цілої інфраструктури (5 вузлів та ініціалізація баз даних) тепер виконується однією командою замість ручного запуску серверів у різних вікнах терміналу.
- Безпечна мережева взаємодія: Контейнери об'єднані у віртуальну закриту мережу. Мікросервіси звертаються один до одного за внутрішніми DNS-іменами (наприклад, `http://service-warehouse:8082`), що приховує внутрішню топологію системи від зовнішнього світу і підвищує безпеку.

3. Потенційні ризики

Незважаючи на значні переваги, ускладнення архітектури вносить нові точки відмови:

- Проблема неактуальних даних: Існує ризик розсинхронізації між базою даних та Redis. Якщо під час оновлення місткості складу (PUT) транзакція в БД пройде успішно, але через мережевий збій команда

очищення кешу не виконається, клієнти продовжуватимуть бачити застарілі дані цілу годину.

- Стартові стани перегонів: Як було виявлено на практиці, асинхронний запуск контейнерів призводить до того, що сервери Node.js намагаються підключитися до PostgreSQL або синхронізувати таблиці до того, як СУБД повністю ініціалізується. Це вимагає додаткових механізмів очікування.
- Ускладнення спостережливості: Логи системи тепер розпорошені по 5 різних контейнерах. У разі виникнення помилки при міжсервісному виклику (наприклад, сервіс посилки не може зв'язатися з сервісом складів), пошук першопричини стає складнішим і вимагає централізованого збору логів.

Висновок:

У ході виконання роботи архітектуру розробленої мікросервісної системи було успішно оптимізовано та підготовлено до розгортання в умовах Production-середовища.

Головним досягненням стало впровадження in-memory сховища Redis, що дозволило суттєво підвищити продуктивність застосунку: швидкість обробки частих запитів на читання зросла в десятки разів, а навантаження на основну базу даних PostgreSQL пропорційно знизилося. При цьому, завдяки налаштованій політиці інвалідації, забезпечено сувору консистентність кешованих даних.

Крім того, інфраструктуру проєкту було повністю контейнеризовано за допомогою технології Docker. Завдяки інструменту оркестрації Docker Compose вдалося об'єднати 5 незалежних вузлів (3 мікросервіси, бази даних та кеш) у єдину віртуальну мережу з автоматичним розгортанням. У результаті створено надійну, масштабовану та ізольовану систему, яка гарантує ідентичну працездатність на будь-якому обладнанні, незалежно від локальних налаштувань хостової операційної системи.